

Roll No.:

END SEMESTER EXAMINATION DECEMBER 2025

Name of the Program: B. Tech. Semester: III
 Name of the Course: Object Oriented Programming with C++ Course Code: TCS-307
 Time: 3 Hours Maximum Marks: 100

Note:

- All questions are compulsory.
- Answer any two sub questions among a, b & c in each main question
- Total marks in each main question are twenty.
- Each question carries 20 marks

Q1	(20Marks)	What are the main limitations of procedural programming that led to the development of Object-Oriented Programming in C++? Describe a real-world scenario (e.g., a library system, vehicle management, or student database) where procedural programming would be difficult to manage, and show how OOP principles make it easier.	CO1
(a)		You are given the two strings <code>string1</code> and <code>string2</code> . Write a program in C++ to remove those characters from the first <code>string(string1)</code> which are present in the second <code>string(string2)</code> . Both strings are different and contain only lowercase characters. Assume the size of the first string is always greater than the size of the second string i.e. <code>(string1) > (string2)</code> . Sample Input: <code>string1 = "occurrence" and string2 = "car"</code> Sample Output: "ouene"	CO1
(b)		1. What are default arguments in C++? Explain how they help in function overloading and code reusability. 2. Write an inline function <code>sumArray(int* arr, int size)</code> that returns the sum of elements in a dynamically allocated integer array using pointer arithmetic. Demonstrate dynamic allocation and deallocation of the array.	CO1
(c)		Design a C++ program that manages employee records using a class <code>Employee</code> with constructors (including a copy constructor) and a <code>display()</code> function. The program should dynamically allocate an array of <code>Employee</code> objects, allow the user to input and display details, then create a deep copy of the array using the copy constructor. Modifying the copied array should not affect the original, demonstrating correct deep copy behaviour.	CO2
(a)		Create a class <code>Complex</code> to represent complex numbers with: Data members: float real, float imag. Use constructors for initialization and a friend functions to overload the following operators: <code>+</code> <code>-</code> <code>*</code> <code>/</code> for addition of two complex numbers, <code>-</code> <code>-</code> for subtraction, <code>*</code> <code>*</code> for multiplication. Perform the following tasks: Read two complex numbers from	CO3
(b)			

		the user. Perform addition, subtraction, and multiplication using overloaded operators. Display the results in proper format (e.g., 3 + 2).	
(c)		1. Explain how access specifiers (private, protected, public) help in implementing data hiding in C++. 2. What is the purpose of the scope resolution operator (<code>::</code>) in C++? Explain the different uses of it using the programming examples.	CO2
Q3		Problem of ambiguity can be resolved by virtual base class. Explain the scenario where the concept of virtual base class is used along with its significance in real-world.	CO3
(a)		Design a C++ program that demonstrates multilevel inheritance and constructor/destructor behavior using three classes: <code>Machine</code> $\rightarrow$ <code>Vehicle</code> $\rightarrow$ <code>Car</code> . Each class should initialize its own data members using constructor initialization lists and print messages when constructed or destroyed.	CO3
(b)		In main (), create and copy a <code>Car</code> object to show the order of constructor, destructor, and copy constructor calls, highlighting how constructor chaining works across multilevel inheritance levels.	CO3
(c)		Imagine a publishing company that markets both book and audiocassette versions of its works. Create a class publication that stores the title (a string) and price (type float) of a publication. From this class derive two classes: <code>book</code> , which adds a page count (type int), and <code>tape</code> , which adds a playing time in minutes (type float). Each of these three classes should have a <code>getData()</code> function to get its data from the user at the keyboard, and a <code>putdata()</code> function to display its data.	CO4
Q4		Write a main () program to test the book and tape classes by creating instances of them, asking the user to fill in data with <code>getData()</code> , and then displaying the data with <code>putdata()</code> .	
(a)		Design a C++ program that demonstrates method overloading and overriding using a <code>Shape</code> base class and derived classes <code>Circle</code> and <code>Rectangle</code> . The <code>Shape</code> class includes overloaded <code>display()</code> methods and a virtual <code>area()</code> function. The derived classes override both <code>display()</code> and <code>area()</code> to provide shape-specific behavior. In main(), use base class pointers or references to show runtime polymorphism when calling <code>area()</code> .	CO5
(b)		Demonstrate the use of virtual function with appropriate programming example. Also explain the need of pure virtual function in context of late binding.	CO5
(c)		"Constructor can't be inherited, although a derived class constructor can call the base class constructor". Justify this statement with an appropriate programming example.	
Q5		Design a C++ program to manage employee records stored in a binary file (<code>employees.dat</code>). Each record includes an employee ID, name, department, and salary. The program must support: Adding new employee records, searching for a record by ID using random file access (seek operations), Updating a salary directly within the file, Deleting records logically (marking inactive, not removing physically), and Displaying only active records at startup.	CO6
(a)		Create a C++ program that simulates a banking system using custom exception classes. Implement a <code>BankAccount</code> class with deposit and withdrawal functions that can throw specific exceptions: <code>InvalidAmountException</code> for non-positive deposits/withdrawals, <code>InsufficientFundsException</code> for withdrawals exceeding balance, and <code>AccountNotFoundException</code> for operations on non-existing accounts.	CO6
(b)		Use a base class <code>BankException</code> and handle all exceptions using appropriate try-catch blocks in main () to display clear error messages.	
(c)		Explain the main components of the Standard Template Library (STL) in C++. Discuss the roles of containers, iterators, algorithms, and describe how they work together to support generic programming. Provide examples to illustrate your explanation.	CO6

